Day 154: Collaborative Filtering: User-Based and Item-Based on a Real Ratings Matrix

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 1 | Complete study guide

Watch: Collaborative Filtering : Data Science Concepts (ritvikmath) | Recommender Systems 4 Item Item Collaborative Filtering (From Languages to Information)

Script: `day154_collaborative_filtering.py` · every number in this guide is copied by program from `day154_run_output.txt`, the real output of that script.

Week 23 turns from forecasting a series to recommending: guessing how much a person will like a film they have not seen, from the ratings that they and everyone else have given. The data is MovieLens 100K (100,000 ratings by 943 users of 1682 films), a table only 6.3% filled. Item-based collaborative filtering scored 0.9153 stars of error (RMSE) on held-out ratings, user-based 0.9353, and a model with no neighbours at all, just an average plus a user bias and a film bias, 0.9401. A control that kept the similarity values but gave them to random film pairs scored 0.9748, worse than no neighbours. Turned into top-10 lists, the item-based predictions found liked films in 0.0417 of slots, while simply recommending the most-rated films found them in 0.1381.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

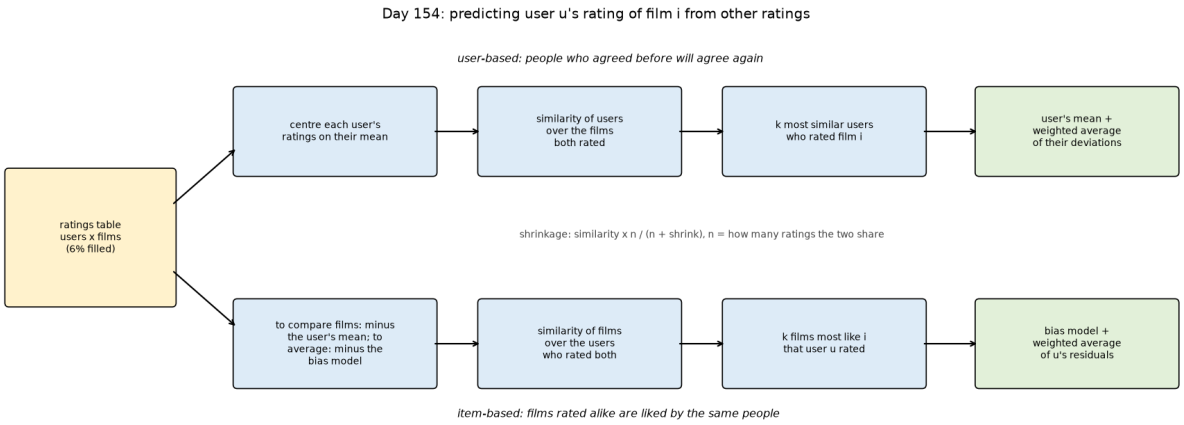
A **recommender system** suggests items (films, songs, products) a person is likely to want. **Collaborative filtering** does it from the record of who liked what, and nothing else: no genres, no plot, no prices. **User-based** collaborative filtering finds people whose past ratings agree with yours and uses their opinion of a new film. **Item-based** finds films that were rated alike and uses your opinion of those.

The week's plan: today, neighbours; Day 155, matrix factorization (SVD); Day 156, content-based filtering (using the films' genres); Day 157, hybrids; Day 158, implicit feedback and cold start; Day 159, ranking metrics (precision@k, recall@k, NDCG); and Day 160 builds a neural recommender from the week's parts.

REAL-WORLD SCENARIO

You ask a friend whose taste has matched yours for years what to watch tonight: that is user-based. A shop says 'people who bought this kettle also bought this toaster': that is item-based, the idea Amazon made famous.

SYSTEM DIAGRAM



User-based (top) and item-based (bottom) collaborative filtering, predicting one user's rating of one film.

Today's part for the week's build

`day154_exercise.py` is today's parts (the data loaders, the split, the ratings table, the bias model, the similarity with shrinkage, the two neighbour predictors and the two error measures), checked by hand: the dataset's sizes, a split, biases on a table built to be exactly additive, a cosine and a user-based and item-based prediction worked on paper.

ACTUAL OUTPUT

```
MovieLens 100K: 100,000 ratings, 943 users, 1,682 films, 19 genres; gives 100,000, 943, 1,682, 19
film 50 (index 49) is Star Wars (1977); gives Star Wars (1977)
split of 10 ratings, 20% held out: 8 kept, 2 out, no overlap, all 10 used; gives 8, 2, 0, 10
biases on a full table that is exactly 3 + user part + film part: rebuilt to within 0.0e+00
films 1 and 4 share users 1-4: (5x1 + 4x1 + 1x5 + 1x4) / (sqrt(25+16+1+1) x sqrt(1+1+25+16)) = 18
    / 43 = 0.4186;
  similarity gives 0.4186 from 4 shared raters; with shrink 4, x 4 / (4 + 4): 0.2093, gives 0.2093
item-based, film C: k 2 gives (0.5 x 1 + 0.25 x -1) / 0.75 = 0.3333, k 1 gives 1; with damping
    0.25, 0.25 / 1.0 = 0.25;
  gives 0.3333, 1.0000, 0.2500
user-based, user 1: (0.6 x 2 + 0.2 x -1) / 0.8 = 1.25 above user 1's mean; gives 1.2500
errors of predictions 1, 2 against 2, 4: RMSE sqrt((1 + 4) / 2) = 1.5811, MAE 1.5; gives 1.5811,
    1.5000
```

Data: MovieLens 100K was collected by GroupLens at the University of Minnesota (F. M. Harper and J. A. Konstan, *The MovieLens Datasets: History and Context*, ACM TiiS 5(4), 2015). Its licence does not allow it to be copied into the repository, so the script downloads it (4.9 MB) to `~/movielens_data` the first time it runs. Everything else is NumPy.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The data: a ratings table that is mostly empty

DEFINITION — start here, no prior knowledge assumed

A **ratings matrix** is a table with one row per user and one column per film; a cell holds a user's rating (1 to 5 stars) of that film, or nothing if they never rated it. It is **sparse**: almost every cell is empty, and filling in the empty cells is the whole job.

A **long tail** means a few items get most of the attention and very many get almost none.

REAL-WORLD SCENARIO

A video shop's ledger of who borrowed what: the new blockbusters are borrowed by hundreds, the old documentaries by two or three. Most customers have borrowed a few dozen of the shop's thousands of films.

ACTUAL OUTPUT

```
100,000 ratings by 943 users of 1682 films; the 943 x 1682 table has 1,586,126 cells
  filled: 0.0630 of the cells (6.3%); the rest is what a recommender must guess
  stars:  1: 6,110 (0.061)   2: 11,370 (0.114)   3: 27,145 (0.271)   4: 34,174 (0.342)   5: 21,201 (0.212);  mean 3.530
  ratings per user: fewest 20, median 65, most 737
  ratings per film: fewest 1, median 27, most 583 (Star Wars (1977))
  the most-rated 10% of films (168) hold 0.427 of all ratings; 333 films have fewer than 5
  commonest genres: Drama 725, Comedy 505, Action 251 (a film can have several)
  collected over 215 days, 1997-09-20 to 1998-04-22
Actual output from this lesson's run.
```

What it means

Only 0.0630 of the cells are filled. Ratings lean high (mean 3.530; 4 is the commonest, 0.342 of ratings), a likely reason, not tested here, being that people mostly watch films they expect to like. The tail is long: the most-rated 10% of films hold 0.427 of all ratings, while 333 films have fewer than 5. Every user has at least 20 ratings (fewest 20), because GroupLens removed users with fewer; that makes this dataset easier than a real service, where most users have rated almost nothing (Day 158).

2. A split, and four baselines that use no neighbours

DEFINITION — start here, no prior knowledge assumed

To measure a recommender honestly, some known ratings are hidden (the **test set**), the model learns from the rest, and its guesses for the hidden ones are compared with the truth. A **validation set**, cut from the training part, is used to choose settings, so the test set is touched only once.

RMSE (root mean squared error) and **MAE** (mean absolute error) measure the typical miss in stars; RMSE punishes big misses more. A **baseline** is a simple model any clever one must beat. The best baseline here is the **bias model**: the overall average, plus how generous each user is, plus how well liked each film is.

REAL-WORLD SCENARIO

Before trusting a weather app, compare it with 'tomorrow will be like today'. Before trusting a recommender, compare it with 'everyone rates every film at the average'.

MATHEMATICS

$$pred_{ui} = \mu + b_u + b_i \quad (\mu = \text{mean rating}, b_u = \text{user bias}, b_i = \text{film bias})$$

$$\text{minimise } \Sigma\,(r_{ui} - \mu - b_u - b_i)^2 + \lambda_u\,\Sigma\,b_u^{\,2} + \lambda_i\,\Sigma\,b_i^{\,2}$$

$$\text{film step: } b_i = \Sigma_{u \text{ rated } i}\,(r_{ui} - \mu - b_u)\,/\,(n_i + \lambda_i)\text{; the user step is the same with the roles swapped}$$

SPECIFICATION

Split	the 100,000 ratings shuffled with seed 0: 80,000 training, 20,000 test; 10% of training (8,000) held back as validation
Bias penalties	λ_u, λ_i each from 0, 2, 5, 10, 25, 50; the pair with the lowest validation RMSE
Measured	test RMSE and MAE of four baselines; predictions clipped to 1–5 stars

IMPLEMENTATION

```
def fit_biases(R, M, lam_u, lam_i, iters=20):
    mu = R[M].mean()
    bu = np.zeros(R.shape[0]); bi = np.zeros(R.shape[1])
    nu, ni = M.sum(1), M.sum(0)
    for _ in range(iters):
        bi = ((R - mu - bu[:, None]) * M).sum(0) / np.maximum(ni + lam_i, 1e-9)
        bu = ((R - mu - bi[None, :]) * M).sum(1) / np.maximum(nu + lam_u, 1e-9)
    return mu, bu, bi
```

ACTUAL OUTPUT

```
ratings split at random (seed 0): 80,000 training, 20,000 test; of the training, 8,000 held back for validation
  every test user also has training ratings: True; test films never rated in training: 43
bias penalties chosen on validation: user 5, film 2 (validation RMSE 0.9511)
            test, 20,000 ratings    RMSE     MAE
                 the global mean  1.1263  0.9445
                each user's mean  1.0390  0.8329
                each film's mean  1.0301  0.8205
    mean + user bias + film bias  0.9401  0.7430
without penalties the 'best' film is #1536 (bias +1.91 from 1 rating(s)); with them, #1449 (bias +1.17 from 7)
Actual output from this lesson's run.
```

What it means

Each step helps: the global mean misses by 1.1263, a user's own mean by 1.0390, a film's mean by 1.0301, and both biases together by 0.9401. The penalties matter for films with few ratings: without them the film with the largest bias (+1.91) has just one training rating (bias +1.91 from 1 rating(s)); with them the top film has seven (bias +1.17 from 7), because one rating is no longer enough to stand out. Validation chose penalties user 5, film 2.

A random split of ratings is the textbook setup, but it lets a model learn from a user's later ratings to predict earlier ones; Day 159 compares splitting by time.

WHY CHOOSE THIS?

The bias model is fast, needs no tuning beyond two penalties, and is the offset the item-based model builds on.

WHY NOT THIS? (TRADEOFFS)

It gives every user the same ranking of films (only the film bias differs between films), so it cannot personalise.

3. Similarity, checked by hand, and the small-overlap problem

DEFINITION — start here, no prior knowledge assumed

Similarity scores how alike two films are, from the users who rated both. The **cosine** treats each film's ratings as a list of numbers and measures the angle between the two lists: 1 when they rise and fall together, 0 when unrelated, –1 when opposite. Subtracting an average first (the film's own mean gives **Pearson** correlation; each user's mean gives **adjusted cosine**) removes generosity, so only agreement counts.

Shrinkage multiplies a similarity by n / (n + shrink), where n is the number of shared raters: a similarity built on two people is pulled toward 0, one built on two hundred is left almost alone.

REAL-WORLD SCENARIO

Two strangers who both loved the one film they have in common are not yet 'people with the same taste'. Two friends who have agreed about two hundred films are.

MATHEMATICS

$$sim(a, b) = \Sigma_{u \in U_{ab}} x_{ua} x_{ub} / (\sqrt{\Sigma}_{u \in U_{ab}} x_{ua}^{2} \cdot \sqrt{\Sigma}_{u \in U_{ab}} x_{ub}^{2}), \quad U_{ab} = \text{users who rated both}$$

$$\text{shrunk: } sim(a, b) \cdot n_{ab} / (n_{ab} + shrink), \quad n_{ab} = |U_{ab}|$$

IMPLEMENTATION

```
def similarity(X, M, shrink=0.0):
    Mf = M.astype(float); X2 = X * X
    num = X.T @ X
    den = np.sqrt(X2.T @ Mf) * np.sqrt(Mf.T @ X2)
    S = np.divide(num, den, out=np.zeros_like(num), where=den > 0)
    n = Mf.T @ Mf
    if shrink:
        S = S * n / (n + shrink)
    np.fill_diagonal(S, 0.0)
    return S, n
```

X is 0 wherever nothing was rated, so X^TX already sums over shared users only; the two lengths are restricted to shared users by multiplying the squared entries by the other film's mask.

ACTUAL OUTPUT

```
a 5-user, 4-film example: films 1 and 4 share 4 raters; cosine 0.4186, by hand 0.4186
a full 30 x 6 table, centred by each film's mean: against numpy's corrcoef, largest difference 1.1e-16
films' similarities on the training ratings (residuals from the bias model): 1,413,721 pairs, 0.401 share no rater
  pairs sharing   1-2 raters:   394,036; |similarity| > 0.9 in 0.786, mean |s| 0.892
  pairs sharing   3-9 raters:   272,047; |similarity| > 0.9 in 0.044, mean |s| 0.411
  pairs sharing 10-49 raters:   165,063; |similarity| > 0.9 in 0.000, mean |s| 0.215
  pairs sharing   50+ raters:    16,327; |similarity| > 0.9 in 0.000, mean |s| 0.126
most similar to Star Wars (1977), no shrinkage:
    +1.000  shared raters   1  Mirage (1995)
    +1.000  shared raters   1  MURDER and murder (1996)
    +1.000  shared raters   1  Wedding Bell Blues (1996)
    +1.000  shared raters   1  Next Step, The (1995)
    +1.000  shared raters   1  Nothing Personal (1995)
most similar to Star Wars (1977), shrinkage 100:
    +0.517  shared raters 316  Return of the Jedi (1983)
    +0.446  shared raters 227  Empire Strikes Back, The (1980)
    +0.293  shared raters 236  Raiders of the Lost Ark (1981)
    +0.180  shared raters 107  L.A. Confidential (1997)
    +0.161  shared raters  78  Austin Powers: International Man of Mystery (1997)
Actual output from this lesson's run.
```

What it means

The vectorised similarity matches the cosine worked by hand (cosine 0.4186, by hand 0.4186) and, on a full table, numpy's Pearson correlation (1.1e-16).

On the real data, 0.401 share no rater, and pairs that share only 1–2 raters have |similarity| above 0.9 in 0.786 of cases: with one shared rater the cosine is always exactly ±1. Without shrinkage, the five films 'most like' Star Wars are obscure films sharing a single rater (Mirage (1995) and others). With shrinkage 100 the list becomes Return of the Jedi (1983), Empire Strikes Back, The (1980) and Raiders of the Lost Ark (1981), which is what a film fan would say.

WHY CHOOSE THIS?

Cosine-type similarities are cheap (one matrix product for all 1.4 million pairs) and easy to explain.

WHY NOT THIS? (TRADEOFFS)

They know nothing about pairs that share no rater, and are unreliable for pairs that share few; shrinkage is a patch, and its strength has to be tuned.

4. User-based collaborative filtering

DEFINITION — start here, no prior knowledge assumed

To predict user u's rating of film i: take the users who rated film i, keep the k most similar to u, and average how far each of them rated film i above or below their own mean, weighted by similarity. Add that to u's own mean.

k is the **neighbourhood size**: small k listens to a few close neighbours, large k to many looser ones.

REAL-WORLD SCENARIO

Asking the ten friends whose taste is most like yours, among those who have seen the film, and adjusting for which of them are harsh critics.

MATHEMATICS

$$pred_{ui} = mean_u + \Sigma_{v \in N_k(u;\, i)}\, s_{uv}\,(r_{vi} - mean_v) \,/\, \Sigma_{v \in N_k(u;\, i)}\, s_{uv}$$

$N_k(u; i)$ = *the k users most similar to u among those who rated i, positive similarities only*

IMPLEMENTATION

```
def _weighted(s, d, k, damp=0.0):
    kk = min(k, s.shape[1])
    top = np.argpartition(-s, kk - 1, axis=1)[:, :kk]
    st = np.take_along_axis(s, top, 1); dt = np.take_along_axis(d, top, 1)
    st = np.where(st > 0, st, 0.0)
    w = st.sum(1)
    return np.where(w > 0, (st * dt).sum(1) / np.where(w > 0, w + damp, 1.0), 0.0), (st >
        0).sum(1)
def predict_user_knn(D, M, S, users, items, k):
    out = np.zeros(len(users)); used = np.zeros(len(users), int)
    for ii, q in _groups(items):
        raters = np.flatnonzero(M[:, ii])
        if len(raters) == 0:
            continue
        s = S[np.ix_(users[q], raters)]
        d = np.broadcast_to(D[raters, ii], s.shape)
        out[q], used[q] = _weighted(s, d, k)
    return out, used
```

ACTUAL OUTPUT

```
  user-based: validation RMSE
    shrink      k 5     k 10     k 20     k 40     k 80    k 160
         0   1.0227   0.9839   0.9636   0.9562   0.9565   0.9576
        10   0.9952   0.9651   0.9530   0.9505   0.9526   0.9543
        25   0.9963   0.9645   0.9532   0.9504   0.9516   0.9535
        50   0.9956   0.9664   0.9538   0.9505   0.9513   0.9531
       100   0.9967   0.9670   0.9540   0.9508   0.9514   0.9529
       200   0.9992   0.9666   0.9549   0.9508   0.9515   0.9528
       400   0.9998   0.9673   0.9553   0.9509   0.9515   0.9528
       800   0.9999   0.9675   0.9554   0.9508   0.9516   0.9529
  chosen: k 40, shrink 25 (7.5 s for the grid)
test: RMSE 0.9353, MAE 0.7284; neighbours used: median 40, none for 0.004 of test ratings (they get the user's mean)
Actual output from this lesson's run.
```

What it means

Validation chose k 40, shrink 25; very small neighbourhoods are noisy (k 5 with no shrinkage: 1.0227). On the test set user-based scored RMSE 0.9353, only a little better than the bias model's 0.9401.

WHY CHOOSE THIS?

Intuitive and easy to explain ('people like you liked this'), and it needs no training beyond computing similarities.

WHY NOT THIS? (TRADEOFFS)

User similarities rest on thin overlaps (most pairs of users share few films), they change whenever a user rates something, and a service with millions of users has trillions of pairs.

5. Item-based collaborative filtering, against the baselines and a control

DEFINITION — start here, no prior knowledge assumed

To predict user u's rating of film i: among the films u has rated, keep the k most similar to film i, and average how far u rated each of them above or below the bias model, weighted by similarity. Add that to the bias model's prediction.

A **control** keeps everything the same except the one ingredient being credited. Here the similarity values are shuffled among film pairs: same weights, random neighbours. If the model still worked, the similarities would not be what made it work.

REAL-WORLD SCENARIO

'You gave Fargo five stars and Contact five stars; this film is a lot like both, so you will probably like it more than most people do.'

MATHEMATICS

$$pred_{ui} = b_{ui} + \Sigma_{j \in N_k(i;u)} s_{ij} (r_{uj} - b_{uj}) / \Sigma_{j \in N_k(i;u)} s_{ij}, \quad b_{ui} = \mu + b_u + b_i$$

$N_k(i; u)$ = *the k films most similar to i among those u rated, positive similarities only*

SPECIFICATION

Similarities compared	cosine on raw ratings; Pearson (minus each film's mean); adjusted cosine (minus each user's mean); minus the bias model. The prediction formula is the same for all four.
Grid	k from 5, 10, 20, 40, 80, 160; shrink from 0, 10, 25, 50, 100, 200, 400, 800; chosen on validation RMSE
Control	the chosen similarity matrix with its off-diagonal values shuffled among film pairs (seed 0)
Breakdowns	test RMSE by how many training ratings the user has, and by how many ratings the film has

IMPLEMENTATION

```
def predict_item_knn(D, M, S, users, items, k, damp=0.0):
    out = np.zeros(len(users)); used = np.zeros(len(users), int)
    for uu, q in _groups(users):
        rated = np.flatnonzero(M[uu])
        s = S[np.ix_(items[q], rated)]
        d = np.broadcast_to(D[uu, rated], s.shape)
        out[q], used[q] = _weighted(s, d, k, damp)
    return out, used
```

ACTUAL OUTPUT

```
  similarity on raw ratings (cosine)                   best validation RMSE 0.9459 (k 80, shrink 0)
  similarity on minus the film's mean (Pearson)        best validation RMSE 0.9260 (k 40, shrink 800)
  similarity on minus the user's mean (adjusted cosine)  best validation RMSE 0.9242 (k 40, shrink 800)
  similarity on minus the bias model                   best validation RMSE 0.9264 (k 40, shrink 800)
  item-based, minus the user's mean (adjusted cosine): validation RMSE
    shrink      k 5     k 10     k 20     k 40     k 80    k 160
         0   1.0186   0.9751   0.9505   0.9407   0.9371   0.9369
        10   0.9730   0.9418   0.9299   0.9279   0.9283   0.9286
        25   0.9726   0.9388   0.9274   0.9259   0.9266   0.9269
        50   0.9759   0.9391   0.9270   0.9249   0.9257   0.9259
       100   0.9738   0.9391   0.9270   0.9245   0.9252   0.9253
       200   0.9722   0.9388   0.9274   0.9243   0.9250   0.9250
       400   0.9731   0.9391   0.9272   0.9244   0.9249   0.9249
       800   0.9737   0.9391   0.9274   0.9242   0.9248   0.9248
  chosen: minus the user's mean (adjusted cosine), k 40, shrink 800 (28.0 s for the grids)
  the limit as shrink grows (weights s x n; the 1 / shrink cancels in the weighted average): validation RMSE 0.9242 at k 40
```

ACTUAL OUTPUT

```
                           test, all models    RMSE     MAE
                            the global mean  1.1263  0.9445
                           each user's mean  1.0390  0.8329
                           each film's mean  1.0301  0.8205
                mean + user bias + film bias  0.9401  0.7430
                                 user-based  0.9353  0.7284
                                 item-based  0.9153  0.7183
  control: item-based, shuffled similarities  0.9748  0.7660
item-based neighbours used: median 40; none for 0.003 of test ratings (they get the bias model)
test RMSE by how many training ratings the user has:
     ratings test pairs  bias model  user-based  item-based
        0-49      3,156      0.9831      1.0013      0.9724
      50-149      7,323      0.9373      0.9352      0.9109
        150+      9,521      0.9275      0.9124      0.8990
test RMSE by how many ratings the film has (in all the data):
        0-19      1,017      1.0416      1.0607      1.0557
       20-99      6,022      0.9689      0.9646      0.9487
        100+     12,961      0.9178      0.9106      0.8872
Actual output from this lesson's run.
```

What it means

Centring matters more than anything else: raw cosine reached only 0.9459 on validation, the three centred versions 0.9242–0.9264. Adjusted cosine won narrowly. Its best shrink, 800, is the edge of the grid, but the curve is flat there: as the shrink grows, every similarity approaches s · n / shrink, and the weighted average divides the common 1 / shrink out, so the predictions approach a limit; computed directly, that limit scores validation RMSE 0.9242 at k 40, the same as shrink 800.

On the test set item-based scored 0.9153, the best of the day, against 0.9401 for the bias model and 0.9353 for user-based. The control scored 0.9748: random neighbours with the same weights are worse than no neighbours at all, so the gain comes from which films are chosen as neighbours, not from the averaging.

Neighbours help most where there is most evidence. For users with 150+ training ratings item-based scored 0.8990 against 0.9275 for the bias model; for films with fewer than 20 ratings it scored 1.0557, worse than the bias model's 1.0416. User-based was worse than the bias model for light users (1.0013 against 0.9831): a likely reason, not tested here, is that a user with few ratings has few films to measure similarity on.

WHY CHOOSE THIS?

Film-to-film similarities are widely reported to be more stable than user-to-user ones (not measured here), can be computed ahead of time, and give explanations: 'because you liked Fargo'.

WHY NOT THIS? (TRADEOFFS)

It needs a similarity for every pair of films (1.4 million pairs here; billions in a large shop), it cannot recommend a film nobody has rated, and rare films get unreliable neighbours.

6. From predicted ratings to a top-10 list

DEFINITION — start here, no prior knowledge assumed

A real service does not show predicted ratings; it shows a short list. **Top-N recommendation** ranks every film the user has not rated by its score and shows the best N. **Precision@10** is the share of the 10 shown that the user turned out to like (here: a held-out film they rated 4 or 5); **recall@10** is the share of their liked held-out films that made the list.

Predicting ratings well and ranking well are different jobs; Day 159 studies ranking metrics in depth.

REAL-WORLD SCENARIO

A bookshop's 'staff picks' shelf has room for ten books. It does not matter whether the staff can guess your exact star rating of every book; it matters whether the ten on the shelf include ones you will buy.

SPECIFICATION

Candidates	every film the user did not rate in training (about 1,600 per user)
Hits	test films the user rated 4 or 5; users with at least one such film are averaged
Rankings	item-based predicted rating; the bias model; popularity (training ratings per film); random (seed 0)
Fix tested	dividing by (total similarity + d); d from 0, 0.1, 0.25, 0.5, 1, 2, 4, chosen on validation precision@10

IMPLEMENTATION

```
def precision_at_10(Q, liked):
    who = np.flatnonzero(liked.sum(1) > 0)
    top = np.argsort(-Q, axis=1, kind="stable")[:, :10]
    h = np.take_along_axis(liked, top, 1)[who]
    return (h.sum(1) / 10).mean(), (h.sum(1) / liked[who].sum(1)).mean(), top[who], len(who)
```

ACTUAL OUTPUT

```
for each user, the 10 unrated films with the highest score; a hit is a test film the user rated 4 or 5
916 users have at least one such film (7.9 s to score every unrated pair)
                       ranked by  precision@10  recall@10  ratings per recommended film (median)
       item-based predicted rating        0.0417     0.0288        40
        bias model (mean + biases)        0.0439     0.0310        99
           popularity (most rated)        0.1381     0.1446       346
                           random        0.0084     0.0063        19
user 1 rated 208 films in training; item-based top 5, the rated film that pulled each up most, and the
total similarity behind it (the weighted average divides by this, so a tiny total still gives a full correction):
```

ACTUAL OUTPUT

```
    4.99  Pather Panchali (1955)             (  7 ratings) total 0.122  because of Monty Python and the Hol (5)
    4.97  Close Shave, A (1995)              ( 91 ratings) total 0.931  because of Fargo (1996) (5)
    4.91  Big Bang Theory, The (1994)        (  1 ratings) total 0.001  because of Contact (1997) (5)
    4.85  Casablanca (1942)                  (201 ratings) total 1.785  because of Fargo (1996) (5)
    4.82  Schindler's List (1993)            (242 ratings) total 1.904  because of Return of the Jedi (1983 (5)
a fix: add a constant d to that total (dividing by total + d), so a film with little evidence keeps close to the
bias model; d chosen on validation precision@10 (training without the validation ratings; hits = validation films rated 4-5)
  d 0     validation precision@10 0.0203
  d 0.1   validation precision@10 0.0306
  d 0.25  validation precision@10 0.0305
  d 0.5   validation precision@10 0.0306
  d 1     validation precision@10 0.0295
  d 2     validation precision@10 0.0285
  d 4     validation precision@10 0.0267
test, d 0.1: precision@10 0.0562, recall@10 0.0417, ratings per recommended film (median) 94
  still 2.5 times below popularity (0.1381); test RMSE with d 0.1: 0.9140
Actual output from this lesson's run.
```

What it means

The model that predicts ratings best ranks badly: item-based top-10 lists hit 0.0417, below even the bias model (0.0439), and both are far behind recommending the most-rated films (0.1381). A likely reason for popularity's lead, not tested here, is that people rate what they watch and watch what is popular, so the held-out liked films are themselves mostly popular. Random lists hit 0.0084.

User 1's list shows one mechanism: *The Big Bang Theory* has a single training rating and a total similarity of only total 0.001 to user 1's films, yet the weighted average divides that tiny total out and gives it a full correction and a near-perfect score.

ACTUAL OUTPUT

```
a fix: add a constant d to that total (dividing by total + d), so a film with little evidence keeps close to the
bias model; d chosen on validation precision@10 (training without the validation ratings; hits = validation films rated 4-5)
  d 0     validation precision@10 0.0203
  d 0.1   validation precision@10 0.0306
  d 0.25  validation precision@10 0.0305
  d 0.5   validation precision@10 0.0306
  d 1     validation precision@10 0.0295
  d 2     validation precision@10 0.0285
  d 4     validation precision@10 0.0267
test, d 0.1: precision@10 0.0562, recall@10 0.0417, ratings per recommended film (median) 94
  still 2.5 times below popularity (0.1381); test RMSE with d 0.1: 0.9140
Actual output from this lesson's run.
```

What it means

Adding d to the denominator keeps films with little evidence near the bias model. Validation chose d 0.1; on the test set precision@10 rose to 0.0562, and the recommended films became more popular (median 94 ratings against 40), while RMSE barely moved (0.9140). It is still 2.5 times below popularity.

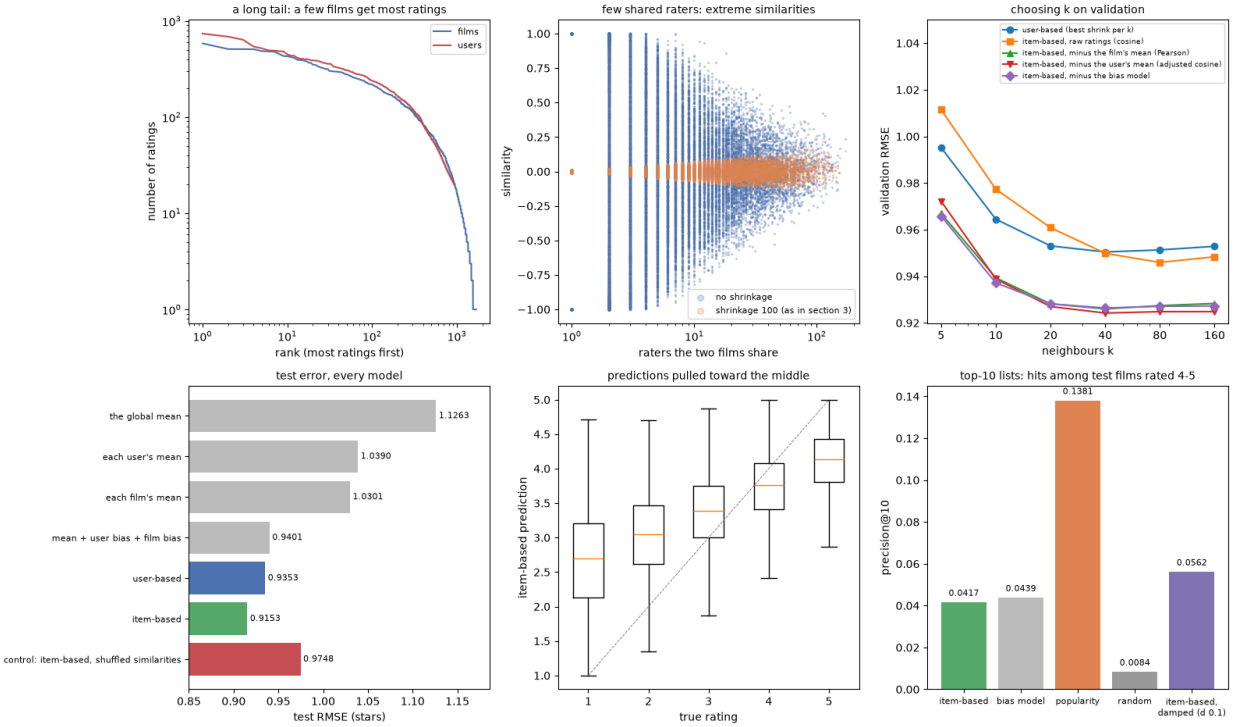
A disclosure: while this section was being built, an earlier draft (with a smaller shrink) printed test precision for several values of d. The value used here is validation's choice, made without the test set.

WHY CHOOSE THIS?

Top-N lists are what users actually see, and precision@10 measures them directly; damping is one line of code.

WHY NOT THIS? (TRADEOFFS)

A ranking can be judged only on the films users chose to rate, which favours popular films; Day 159 looks at that bias and at better ranking measures.



Top: ratings per film and per user (log scales); similarity against shared raters, before and after shrinkage 100; validation RMSE against k. Bottom: test RMSE of every model; item-based predictions by true rating (pulled toward the middle); precision@10 of each top-10 ranking.

Interview preparation — technical questions

Q: Why centre ratings before computing similarity?

A: Users differ in generosity; centring removes that so only agreement counts. Raw cosine reached 0.9459 validation RMSE, the centred versions 0.9242 to 0.9264.

Q: What is shrinkage and why is it needed?

A: Multiplying a similarity by n / (n + shrink). Pairs with one shared rater always have similarity ±1; without shrinkage Star Wars's 'nearest' films were obscure films sharing one rater.

Q: User-based or item-based?

A: Here item-based was better (0.9153 against 0.9353) and it is the usual choice in practice: film similarities can be precomputed and give explanations ('because you liked Fargo').

Q: How do you know the neighbours, not the averaging, help?

A: Shuffle the similarities among pairs: same weights, random neighbours. That control scored 0.9748, worse than the bias model's 0.9401.

Q: Why can a good rating predictor make a bad top-10 list?

A: The weighted average ignores how much evidence stands behind a score, so films with one rating can top the list; item-based precision@10 was 0.0417 against 0.1381 for popularity.

Interview preparation — theoretical questions

Q: What does 'sparse' mean for a ratings matrix, and why does it matter?

A: Most cells are empty (93.7% here). Similarities rest on few shared ratings, many pairs share none, and every method has to cope with missing data.

Q: What is the cold-start problem?

A: A new user or film has no ratings, so collaborative filtering has nothing to compare; here 43 test films had no training ratings. Days 156 and 158 address it.

Q: Why is RMSE not enough to evaluate a recommender?

A: Users see a ranked list, not ratings; the ranking also depends on which films are scored at all and on popularity. Day 159 covers ranking metrics.

Q: What is the bias model and why is it a strong baseline?

A: Mean + user bias + film bias. Much of a rating is the user's generosity and the film's general quality; it alone took RMSE from 1.1263 to 0.9401.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Collaborative filtering predicts ratings from ratings alone. Item-based neighbours, with centred and shrunk similarities, beat user-based neighbours and a strong bias model, and a shuffled-similarity control shows the gain comes from choosing the right neighbours. But accuracy on ratings did not turn into good top-10 lists: popularity beat every personal ranking, which sets up the rest of the week.

- Test RMSE: bias model 0.9401, user-based 0.9353, item-based 0.9153, control 0.9748.
- One shared rater means a similarity of ±1; shrinkage turned Star Wars's neighbours from obscure films into Return of the Jedi (1983).
- Neighbours help heavy users and popular films; for rare films item-based (1.0557) lost to the bias model (1.0416).
- Top-10 precision: item-based 0.0417, damped 0.0562, popularity 0.1381.

Suggested watch order, if starting from zero

- 1. Collaborative Filtering : Data Science Concepts (ritvikmath) — what collaborative filtering is, with a worked example.
- 2. Recommender Systems 4 Item Item Collaborative Filtering (From Languages to Information) — item-item collaborative filtering, the method that won today, from a course lecture.

Day 155 preview

- **Matrix factorization.** Instead of comparing users or films directly, describe each by a short list of hidden numbers, learned so that their products rebuild the known ratings. Truncated SVD and its regularised relative (Funk SVD) are built from scratch and compared with today's neighbours.